Voice Scribe v4
Real-Time Streaming Ingestion

Johnraymond Yanez

Voice Scribe

July 31, 2026

Overview

- **The Latency Problem** — where the 1.2–1.5 s actually goes
- **Streaming Serving** — overlap inference with speech
- **Why Whisper Cannot Stream** — and the policy layer that fakes it
- **LocalAgreement** — the serving loop, instance to instance
- **Session Architecture** — protocol, ring buffer, decode ticks
- **Tunables & Targets** — five knobs, four measured numbers
- **Hardware Cost** — v3 vs. v4, the compute premium justified
- **Serving Vocabulary** — what this milestone teaches
- **Plan** — v4.0–v4.2, testing, gate

v2.1 "Streaming" Is Output-Only

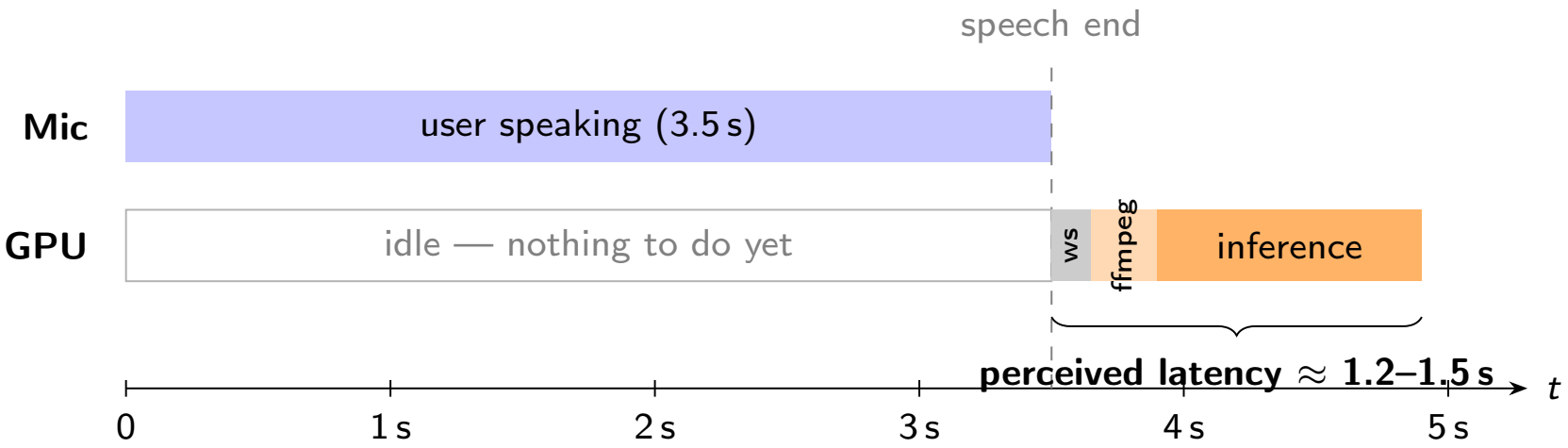
The existing WebSocket path streams *results out*, never *audio in*:

1. Client VAD waits for a **complete utterance** (`onSpeechEnd`)
2. WAV encode → **new WebSocket per utterance** → whole blob in one message
3. Server writes a temp file → ffmpeg transcode → **full-clip inference**
4. Only the *segments* stream back incrementally

The observation

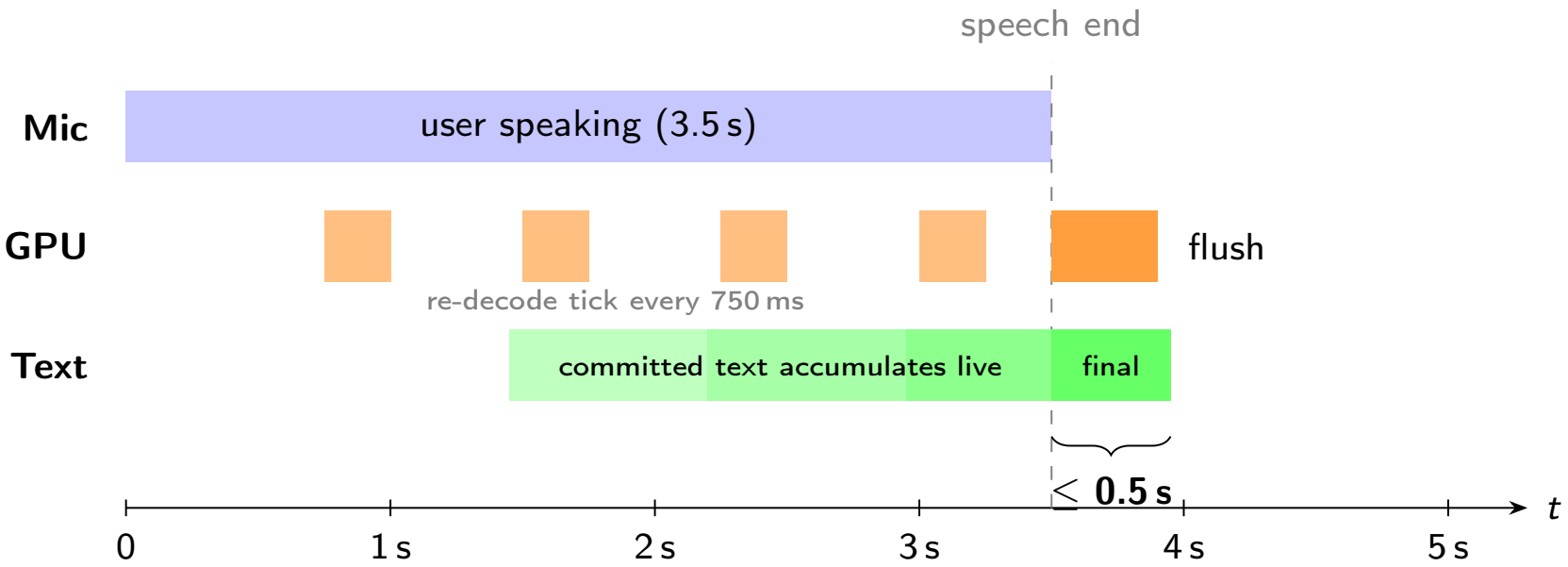
Every stage begins *after* the user stops speaking. The GPU is idle for the entire duration of speech — the one window in which inference could have been free.

Current Serving Timeline



All useful work is serialized *after* speech end. Inference time scales with utterance length, so longer utterances feel *worse*, not better.

Streaming Serving Timeline



Inference overlaps speech. By speech end, most of the transcript is already committed; the flush decodes only the short uncommitted tail. Perceived latency is now *independent of utterance length*.

Why Whisper Cannot Stream Natively

Whisper is an offline model. Encoder–decoder over a 30 s window; you cannot feed it one frame and get one token. Feeding it a growing buffer naïvely gives a *different* hypothesis every pass — text that rewrites itself is unusable for dictation.

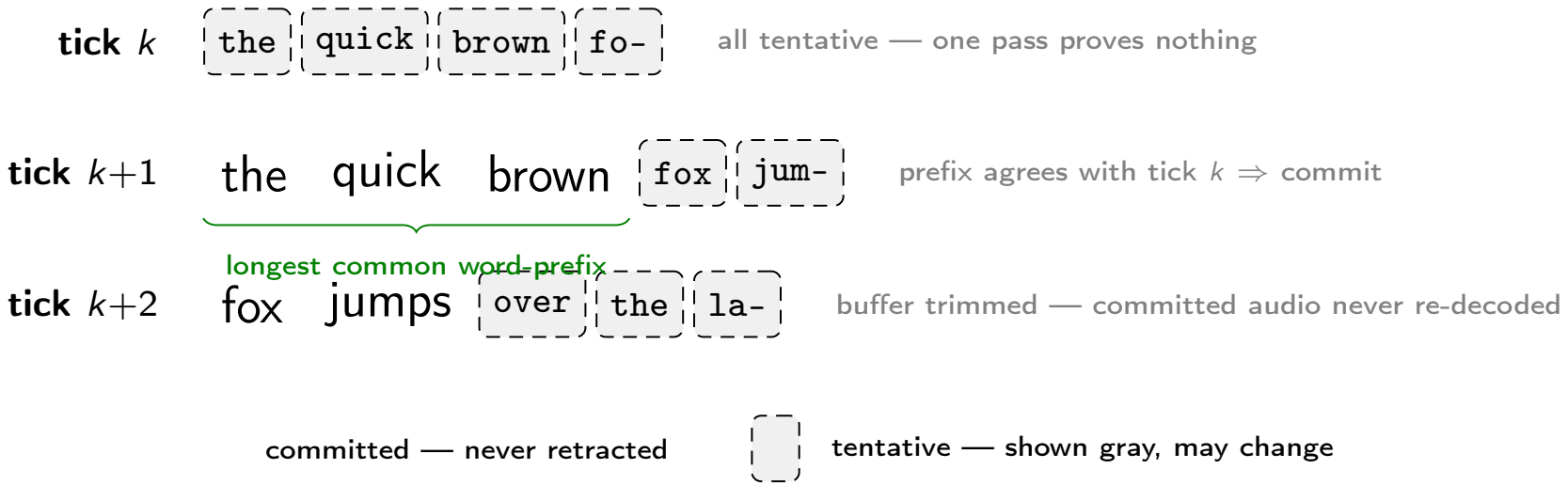
Two ways out:

1. Switch to a natively-streaming architecture (transducer / RNN-T, Kyutai STT) — *rejected*: abandons the proven `large-v3` accuracy stack
2. Keep the model, add a **serving policy layer**: paced re-decode of a rolling buffer + a rule for which prefix is stable enough to commit — *chosen*

The key asymmetry

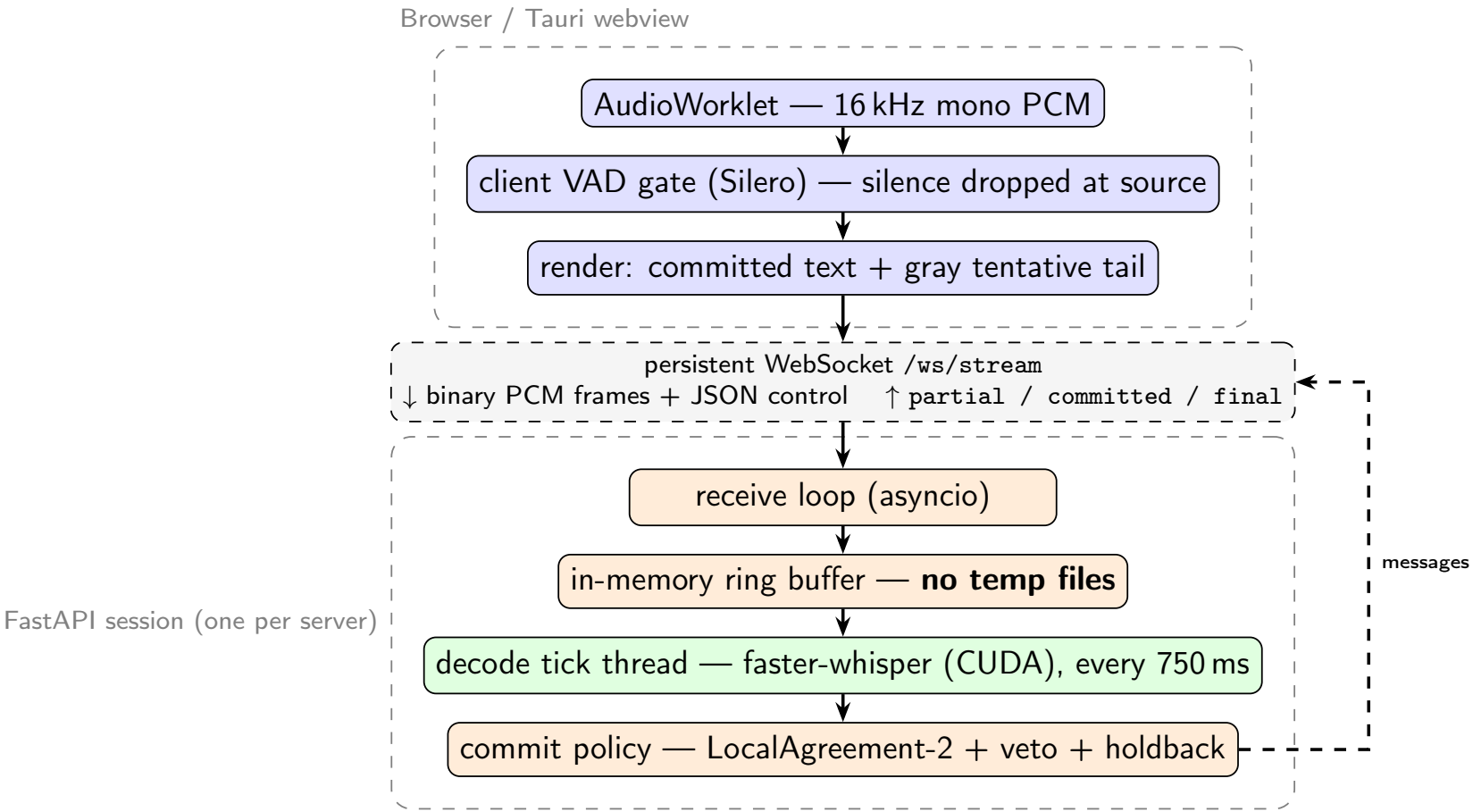
In a *batch* pipeline, overlapping windows are equally informed — you stitch or blend them. In a *streaming* pipeline every new decode sees strictly more context: the newest hypothesis **supersedes** the old one. The only question: *which prefix is stable enough to commit?*

LocalAgreement — Instance to Instance



Agreement-across-time replaces confidence stitching. Confidence enters only as a *veto*: an agreed prefix below `COMMIT_LOGPROB_FLOOR` is held (two passes can hallucinate identically on a mid-word buffer edge). A flip-flopping region is force-committed after `MAX_HOLDBACK_TICKS` — latency is bounded, never hostage.

Session Architecture



The Five Tunables

Parameter	Meaning	Initial
`DECODE_TICK_MS`	Re-decode pacing; main latency/compute dial. Commit lag $\approx$ 2 ticks.	750 ms
`COMMIT_LOGPROB_FLOOR`	Veto: agreed prefix below this `avg_logprob` is not committed.	-1.0
`NO_SPEECH_MAX`	Segments above this `no_speech_prob` never enter the agreement comparison.	0.6
`MAX_HOLDBACK_TICKS`	Force-commit ceiling for flip-flopping regions; bounds worst-case latency.	6
`MAX_BUFFER_S`	Uncommitted-buffer cap; slide with committed left-context before Whisper's 30 s window.	28 s

Plus one non-tunable rule: **VAD flush** — speech end is ground truth. Decode the remaining buffer once, finalize everything. This bounds finalization latency in practice.

The trade, stated honestly

Each second of audio is decoded 3–5×: compute is spent to buy latency. Right trade on a dedicated local GPU — but *measured and reported*, not assumed.

Latency Targets — the Gate Is a Measurement

Metric	Today	v4 target
First partial visible (from speech onset)	n/a	$\leq 1.5\,\mathrm{s}$
Final transcript (from speech end)	$\sim$1.2–1.5 s	$\leq 0.5\,\mathrm{s}$
Committed-text retractions	n/a	0 (by construction)
GPU re-decode overhead	$1\times$	measured, reported

Measurement harness ships with the endpoint:
`ops/scripts/stream_bench.py` replays a WAV over `/ws/stream` at real-time pace; reports first-partial latency, finalization latency, commit-lag distribution, retraction count.

Note : tuning without instrumentation is guessing. The harness is v4.0 scope, not an afterthought — same discipline as v1's smoke-test latency record.

Hardware Cost Justification — v3 vs. v4

Streaming spends compute to buy latency. The premium is *quantified*, not asserted: `stream_bench` runs the **same audio** through both paths and publishes the comparison.

Measured per run	**v3 one-shot (baseline)**	**v4 streaming**
Decode-seconds per audio-second	$\sim 1\times$	expected 3–5$\times$
GPU duty cycle (`nvidia-smi`)	idle until speech end	sustained during speech
Peak VRAM	measured	measured
Per-tick decode time (RTF)	n/a	must stay < 1
Finalization latency	$\sim$1.2–1.5 s	$\leq$ 0.5 s

The justification, stated as a sentence

"An $X\times$ compute premium bought Y ms of perceived latency." X outside the 3–5$\times$ envelope is a tuning finding (tick too fast, buffer trim broken) — not a number to quietly accept. Per-tick RTF ≥ 1 means the buffer grows without bound: a correctness failure, not a performance quirk.

What This Milestone Teaches (Field Vocabulary)

Term	Where it appears in v4
Session	One connection = one stateful stream with its own buffer and decode loop
First-token latency	First-partial latency — the streaming world's headline metric
Finalization latency	Speech end → final text; what the user actually feels
RTF (real-time factor)	Decode time ÷ audio duration; must stay < 1 per tick or the buffer grows unboundedly
Commit / stability policy	LocalAgreement + veto + holdback — the quality/latency contract
Backpressure	What happens when decode ticks fall behind frame arrival (skip ticks, never queue unboundedly)
Graceful degradation	Stream fails → one-shot WS → HTTP POST; three-deep fallback chain

Contrast worth knowing: LLM serving batches *across users* (continuous batching); a local voice tool batches *across time* for one user. Same discipline — schedulers, budgets, SLOs — different axis. Streaming ASR policy layers, session management, and latency instrumentation are exactly the skills the voice-systems field hires for.

Sub-Milestones

Milestone	Deliverable
v4.0	Backend session loop: `/ws/stream`, ring buffer, decode ticks, LocalAgreement commit, VAD flush. Driven by `stream_bench` — no frontend changes
v4.1	Frontend: persistent session in continuous mode, partial-vs-committed rendering. `state-machine.mmd` and `privacy-data-flow.mmd` updated *before* code
v4.2	Voice command fast path on committed text + tuning pass against `stream_bench` numbers

Testing folded into each sub-milestone:

- Commit policy as **pure functions** — unit-tested on synthetic hypothesis sequences, no GPU, no WebSocket
- Session tests with the existing model-mocking pattern
- vitest: partial rendering, retraction, fallback, teardown

Design review checkpoint before v4.0 code: fresh-context adversarial pass on protocol + commit policy.

Gate

- `stream_bench`: all four targets met on GPU with `large-v3`
- Three-minute live dictation: committed text never visibly changes; tentative tail flickers but never survives incorrectly
- Final segments match one-shot path quality on the same audio
- Kill the server mid-stream → graceful fallback to one-shot path
- VRAM and process table clean after session close
- All existing tests green; one-shot and HTTP paths unchanged

Why v4 stays inside the capture boundary

v4 changes *when* inference happens, not *what* the product does. No transformation, no routing, no instrumentation of user content. The streaming path removes temp files entirely — zero-persistence gets *stronger*. The downstream-product boundary (roadmap, Adjacent Projects) is unchanged.